ASSIGNMENT 1

Study of Deep Learning Packages

SPPU • BE Information Technology • Lab Practice IV

Aim

To study and document the distinct features and functionality of four commonly discussed deep learning packages: TensorFlow, Keras, Theano and PyTorch.

What this assignment covers

- What a deep learning framework does
- Features and functionality of each package
- A side-by-side comparison
- Typical use cases, strengths and limitations
- Short viva questions with answers

Prepared as practical-exam study notes

1. Introduction

Deep learning is a branch of machine learning that uses artificial neural networks with multiple layers to learn patterns from data. Deep learning frameworks provide ready-to-use tools to build, train, evaluate and deploy these neural networks.

A framework generally helps with tensor operations, automatic differentiation (calculating gradients), neural-network layers, loss functions, optimizers, GPU acceleration, data loading and model deployment.

2. TensorFlow

Developed by: Google. **Type:** Open-source machine-learning and deep-learning framework.

Distinct features

• Supports tensor computation and automatic differentiation.
• Offers high-level model-building APIs as well as lower-level operations.
• Supports CPU, GPU and distributed training across multiple devices.
• Provides deployment options for servers, browsers, mobile and edge devices through its ecosystem.
• Integrates with tools for data pipelines, model serving and production monitoring.

Functionality

TensorFlow is used to define neural-network computations, train models using optimizers and loss functions, evaluate performance, save trained models and deploy them. Its ecosystem includes tools such as TensorFlow Lite for mobile/edge deployment and TensorFlow.js for JavaScript environments.

Example

A TensorFlow tensor can represent a batch of images, while layers and optimizers can be combined to train a CNN for image classification.

Strengths: broad production ecosystem, hardware acceleration and deployment choices.
Considerations: its ecosystem can feel large to beginners; some workflows require understanding tracing, graphs or deployment-specific formats.

3. Keras

Type: High-level deep-learning API designed to make neural-network development easier.

Distinct features

• Simple, readable and consistent API.
• Supports rapid prototyping and experimentation.
• Provides built-in layers, losses, metrics, optimizers and training utilities.
• Supports common model-building styles such as Sequential and Functional models.
• Current Keras 3 can work with TensorFlow, JAX or PyTorch backends; older material often describes Keras mainly as a TensorFlow interface.

Functionality

Keras lets developers create a neural network by arranging layers, select a loss function and optimizer, and train the model with methods such as compile() and fit(). It reduces the amount of low-level code needed for common deep-learning tasks.

Example

A Sequential Keras model can stack Flatten, Dense and Dropout layers to classify handwritten digits.

Strengths: beginner-friendly, concise code and fast experimentation.
Considerations: highly customized operations may require lower-level backend APIs.

4. Theano

Developed by: The University of Montreal's MILA research community. **Type:** Python library for numerical computation and symbolic mathematical expressions.

Distinct features

• Represents mathematical computations symbolically and compiles them for execution.
• Can optimize computation graphs before execution.
• Supports automatic differentiation.
• Historically supported efficient CPU and GPU numerical computation.
• Influenced the development of early deep-learning tools and frameworks.

Functionality

Theano was used to define mathematical expressions, compile functions and calculate gradients efficiently. It was also used as a computational backend by older versions of high-level libraries, including Keras.

Current status: Theano is a legacy framework and is no longer actively developed as a mainstream deep-learning platform. It is important historically, but it is generally not the first choice for a new project.

Strengths: symbolic computation, graph optimization and historical research importance.
Limitations: discontinued/legacy status and a much smaller modern ecosystem.

5. PyTorch

Developed by: Facebook AI Research (now Meta AI and the wider PyTorch community). **Type:** Open-source machine-learning framework.

Distinct features

• Uses tensors and automatic differentiation through its autograd system.
• Eager execution makes operations run immediately, which is convenient for debugging.
• Supports dynamic model construction and flexible control flow.
• Provides GPU acceleration and distributed-training capabilities.
• Has a broad ecosystem for computer vision, natural language processing and research.

Functionality

PyTorch is used to create neural-network modules, run forward passes, calculate loss, compute gradients with autograd and update parameters using optimizers. Developers typically write an explicit training loop, giving them detailed control over the process.

Example

A PyTorch model can be created by subclassing torch.nn.Module and defining its forward() method.

Strengths: flexible programming model, debugging experience and strong research ecosystem.
Considerations: a custom training loop can involve more code than a high-level Keras workflow.

6. Comparison of the Packages

Parameter	TensorFlow	Keras	Theano	PyTorch
Main role	Full ML/DL framework	High-level DL API	Symbolic computation library	Full ML/DL framework
Ease of use	Moderate	High	Low to moderate	Moderate to high
Execution model	Eager + graph/tracing	Depends on backend	Symbolic graph compilation	Primarily eager/autograd
Automatic differentiation	Yes	Through backend	Yes	Yes
GPU support	Yes	Through backend	Historically yes	Yes
Typical use	Production and deployment	Rapid model development	Legacy research/code	Research and production
Current relevance	Widely used	Widely used	Legacy/discontinued	Widely used

Note: Keras is an API rather than a direct one-to-one equivalent of TensorFlow or PyTorch. It can use different backends in current versions. Theano's role is mainly historical today.

7. When should each be used?

• **TensorFlow:** when a project needs a broad ecosystem and established deployment options, including mobile or browser deployment.
• **Keras:** when you want to build and test standard neural-network architectures quickly with readable code.
• **Theano:** mainly when maintaining or studying older research code that depends on it.
• **PyTorch:** when you need flexible model development, custom training logic, research experimentation or production model training.

8. Important Terms

Tensor: a multidimensional array used to store data and parameters.
Layer: a neural-network component that transforms its input.
Loss function: measures the difference between predictions and target values.
Optimizer: updates model parameters to reduce the loss.
Backpropagation: computes gradients of the loss with respect to model parameters.
Automatic differentiation: calculates derivatives by tracking operations performed during computation.
GPU acceleration: uses a graphics processing unit to speed up suitable numerical operations.

9. Conclusion

TensorFlow and PyTorch are full-featured deep-learning frameworks. Keras provides a high-level interface that makes model development simpler and faster. Theano was an influential early framework for symbolic numerical computation, but it is now a legacy technology. The best choice depends on the project's requirements, developer experience, flexibility needs and deployment environment.

10. Viva Questions and Short Answers

1. What is a deep-learning framework?
A software library that provides tools to build, train, evaluate and deploy neural-network models.

2. What is TensorFlow?
An open-source machine-learning framework developed by Google for tensor computation and building/deploying ML models.

3. What is Keras?
A high-level deep-learning API that simplifies building and training neural networks.

4. Is Keras the same as TensorFlow?
No. Keras is an API. It commonly works with TensorFlow and current Keras 3 can also use JAX or PyTorch backends.

5. What is Theano?
A Python library for symbolic mathematical computation, graph optimization and automatic differentiation. It is now a legacy project.

6. What is PyTorch?
An open-source deep-learning framework known for tensor operations, autograd and flexible eager execution.

7. What is the difference between TensorFlow and PyTorch?
Both are full frameworks. PyTorch is known for its eager, Python-friendly workflow; TensorFlow offers a broad ecosystem and multiple graph/deployment tools. Both support automatic differentiation and GPU computing.

8. What is automatic differentiation?
A technique used by frameworks to calculate derivatives of computations, enabling gradient-based learning.

9. What is a computational graph?
A representation of operations and their dependencies, used to describe and execute a computation.

10. Which package is easiest for beginners?
Keras is often easiest for beginners because it offers a concise, high-level API.

11. Which package is no longer actively developed?
Theano is a legacy framework and is no longer actively developed as a mainstream platform.

12. Can TensorFlow and PyTorch use a GPU?
Yes. Both support GPU acceleration when compatible hardware and software are configured.

13. What is an optimizer?
An algorithm that adjusts model parameters using gradients to reduce the loss function.

14. What is the main advantage of Keras?
It makes neural-network development faster and more readable by providing high-level building blocks.

15. Does choosing a framework guarantee model accuracy?
No. Accuracy depends on data quality, preprocessing, architecture, training, hyperparameters and evaluation.

Exam tip

If asked to compare the four packages, remember this summary: TensorFlow = broad framework and deployment ecosystem; Keras = simple high-level API; Theano = symbolic computation and legacy research; PyTorch = flexible eager execution and autograd.

Reference: SPPU-BE-IT-DL-ASSIGNMENTS repository, Assignment_1.txt and repository README. This handout expands the listed features into exam-oriented notes.

ASSIGNMENT 1 – CODE EXAMPLES

The following small examples demonstrate the basic workflow of each package. They use a simple classification task so the main API differences are easy to compare.

1. TensorFlow example

This example creates a small neural network, compiles it, and trains it on randomly generated sample data. In a real experiment, replace the random arrays with a properly prepared dataset.

```
import tensorflow as tf
import numpy as np

# Create sample input: 100 samples, 4 features each
X = np.random.random((100, 4)).astype("float32")
y = np.random.randint(0, 2, size=(100,))

# Build a neural network
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

# Configure training
model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# Train the model
model.fit(X, y, epochs=5, batch_size=16)

# Predict probabilities
print(model.predict(X[:5]))
```

Key points: Sequential stacks layers; Dense creates a fully connected layer; compile() selects the optimizer, loss and metrics; fit() trains the model; predict() returns predictions.

2. Keras example

Keras provides a concise high-level API. This example uses the same sample-data shape and binary classification setup.

```
import numpy as np
from keras import Sequential, Input
from keras.layers import Dense, Dropout

X = np.random.random((100, 4)).astype("float32")
y = np.random.randint(0, 2, size=(100,))

model = Sequential([
    Input(shape=(4,)),
    Dense(16, activation="relu"),
    Dropout(0.2),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.fit(X, y, epochs=5, batch_size=16)
print(model.predict(X[:5]))
```

Key points: Input defines the feature shape; Dropout randomly disables a fraction of activations during training to help reduce overfitting; the final sigmoid unit outputs a probability for the positive class. Keras 3 may use a configured backend such as TensorFlow, JAX or PyTorch.

3. Theano example (legacy)

Theano is discontinued as a mainstream deep-learning framework. This example demonstrates its symbolic computation style rather than a modern model-training workflow. It may require an older compatible Python and NumPy environment.

```
import theano
import theano.tensor as T

# Declare symbolic variables
x = T.dscalar("x")
y = T.dscalar("y")

# Define a symbolic expression
z = (x * y) + x

# Compile the expression into a callable function
calculate = theano.function([x, y], z)

# Execute with actual values
result = calculate(3.0, 2.0)
print(result)  # Output: 9.0
```

Key points: T.dscalar declares symbolic scalar variables; the expression defines a computation graph; theano.function() compiles that graph into a function that can be executed with real values. Unlike the other examples, this is a basic symbolic-computation demonstration, not a neural-network training script.

4. PyTorch example

PyTorch uses tensors, nn.Module to define a model, autograd to calculate gradients, and an optimizer to update parameters.

```
import torch
import torch.nn as nn
import torch.optim as optim

# Sample data: 100 samples, 4 features each
X = torch.rand(100, 4)
y = torch.randint(0, 2, (100, 1)).float()

# Define the neural network
class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(4, 16),
            nn.ReLU(),
            nn.Linear(16, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.layers(x)

model = SimpleNet()
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Training loop
for epoch in range(5):
    predictions = model(X)
    loss = criterion(predictions, y)

    optimizer.zero_grad()  # Clear old gradients
    loss.backward()        # Calculate gradients
    optimizer.step()       # Update model weights

    print("Epoch:", epoch + 1, "Loss:", loss.item())

print(model(X[:5]))
```

Key points: nn.Module defines the model; forward() describes the forward pass; BCELoss measures binary-classification error; backward() computes gradients; optimizer.step() updates the weights.

5. Quick comparison of the code

Package	How training is typically started	Distinctive idea
TensorFlow	model.fit(X, y, ...)	Tensor operations and integrated training/deployment ecosystem
Keras	model.fit(X, y, ...)	High-level, concise model-building API
Theano	Compile a symbolic function	Define and compile symbolic mathematical graphs
PyTorch	Explicit training loop	Autograd and direct control over training steps

Important: These are illustrative starter examples using generated data, not a substitute for the assignment's dataset-based practical. Theano code is included for understanding its package functionality; it may not run in a current Python environment because Theano is legacy software.